

TECHNICAL SHOWCASE

RLHF Rubric Design & Calibration

How I design atomic, verifiable rubrics that teach frontier AI models to build real-world software — and how I calibrate difficulty so models are challenged, not set up to fail.

K. Sriharika | Mercor Vibe Coding Project | Feb – Mar 2026

8–20

Rubric items per task

5 Tags

visual · layout · content ·
interaction · state

3 Levels

Easy / Medium / Hard

100%

Verifiable from screenshot
or code

◆ WHAT RUBRIC DESIGN ACTUALLY IS

Most people think AI annotation means rating responses on a scale. Rubric design is different — it is the act of building the measurement instrument itself.

At Mercor's Vibe Coding Project, the task was to train frontier AI models to build client-side web applications — React, HTML, JavaScript. To do this, someone had to define precisely what a correct application looked like. Not in general terms, but in atomic, verifiable criteria that a vision-language model could evaluate from a single screenshot and a code file.

That is what rubric design is. Every criterion must be:

- Self-contained — evaluable without reading other rubric items
- Task-specific — tied to the unique features of the prompt, not generic
- Verifiable — observable from the screenshot or code, not inferred
- Non-stacked — one behaviour per criterion, not bundled
- Appropriately flexible — allowing valid alternative implementations

Writing rubrics that meet all five of these standards simultaneously — for every single criterion across every task — is a form of precision engineering for data quality.

◆ THE TAG SYSTEM — FIVE LENSES ON APPLICATION BEHAVIOUR

Each rubric item is tagged with exactly one of five tags. The tag determines both what is being evaluated and how it will be verified. This matters because a vision-language model evaluating a screenshot can verify visual and layout criteria directly, but cannot reliably verify application state or user interactions — those require code review.

Tag	Verified By	Evaluates	Example
visual	Screenshot	Element existence, colour, styling, visual emphasis — what can be seen on initial load	"Display the player character as a rectangular red box"
layout	Screenshot or Code	Spatial placement and alignment of elements relative to each other	"Position the character at the middle-left of the playfield on initial load"
content	Screenshot or Code	Exact text, labels, numbers, or data values displayed — independent of position	"Display timer initialised at 00:00:00 on page load"
interaction	Code Review	A user action (click, type, drag) and what immediately happens as a result	"Increase task count by one when Increment Task is clicked"
state	Code Review	Data or application state changing or persisting over time	"Pause the timer without resetting elapsed time when Pause is clicked"

◆ LIVE RUBRIC — TASK COUNTER APPLICATION

Below is a real rubric written for the Task Counter prompt — one of the Vibe Coding tasks at Mercor. The application required a timer (HH:MM:SS), a task counter, and five control buttons. Each row below is a single verifiable criterion as it would appear in the evaluation system.

ID	TAG	CRITERION (action-verb first, specific, self-contained)	WEIGHT
001	visual	Display the timer value formatted as HH:MM:SS on the initial render	major
002	content	Display the timer initialised at 00:00:00 on page load	major
003	visual	Display a numeric task count on the initial render	major
004	content	Display the task count initialised at 0 on page load	major
005	layout	Position the application at the centre of the screen	major
006	visual	Display a Start button on the initial render	major
007	visual	Display a Pause button on the initial render	major
008	visual	Display a Stop button on the initial render	major
009	visual	Display an Increment Task button on the initial render	major
010	visual	Display a Reset Tasks button on the initial render	major
011	interaction	Begin the timer countdown when the Start button is clicked	major
012	state	Increase the timer value once per second while running after Start is clicked	major

013	state	Pause the timer without resetting elapsed time when Pause is clicked	major
014	interaction	Reset the timer display to 00:00:00 when the Stop button is clicked	major
015	interaction	Increase the task count by one when Increment Task is clicked	major
016	state	Reset the task count to 0 when Reset Tasks is clicked	major

This rubric covers every explicit requirement of the prompt, uses all five tag types appropriately, and contains zero stacked or redundant criteria. Each item begins with an action verb and can be evaluated independently.

◆ THE ART OF THE CRITERION — GOOD VS BAD

The single most common failure in rubric writing is vagueness. The rubric guidelines require that every criterion be clear enough for its pass/fail condition to be obvious — no forbidden words like 'correctly', 'properly', or 'adequately'. Below are real examples of transforming bad criteria into good ones, drawn from vibe coding tasks.

Principle	✗ BAD — Fails the Standard	✓ GOOD — Passes the Standard
Specific	✗ Countdown works correctly	✓ Display countdown in MM:SS format in the top-right of the playfield
Non-stacked	✗ Start and pause the timer	✓ Begin countdown when the Start button is clicked
Actionable	✗ Countdown displays in MM:SS format	✓ Display countdown in MM:SS format
Flexible	✗ Use a table element for tasks	✓ Display tasks in a list or table format
Verifiable	✗ Keep track of score internally	✓ Show the current score near the game board
Self-contained	✗ Centered	✓ Centre the application horizontally on the screen

◆ DIFFICULTY CALIBRATION — ENGINEERING PASS RATES

Rubrics define correctness. Difficulty calibration defines how hard it should be to achieve it. At Mercor, every task was assigned a difficulty level — not based on how complex the prompt felt, but on the expected model pass rate: the percentage of rubric criteria a well-performing model should clear.

Difficulty	Pass Rate Target	What This Means in Practice
Easy	70 – 100%	Straightforward layout and UI tasks. Model should pass most criteria. Example: a static Task Counter with timer display and clearly labelled buttons; no dynamic state required.
Medium	40 – 70%	Dynamic behaviour with multiple interacting components. Example: a Flappy Bird clone requiring physics (gravity, jump impulse), collision detection, lives system, score tracking, and timer and model will clear layout and visual criteria but may miss complex state interactions.
Hard	0 – 40%	Complex real-time physics, fluid particle simulations, or multi-agent game logic. Model is expected to fail most criteria and the purpose is to identify the frontier of model capability, not to achieve a high pass rate.

Calibration is not about making tasks easier or harder — it is about ensuring the rubric is fair. A rubric that sets an impossible standard for the model produces no useful training signal. A rubric that is too lenient produces overconfident models. The art is in the balance.

◆ WHEN A RUBRIC IS UNFAIR — AND WHAT TO DO

The evaluation system explicitly required that rubric writers check fairness before submitting. If a rubric was unfair — if it penalised the model for something the prompt did not specify, or required a specific implementation when multiple valid approaches existed — the entire evaluation had to be restarted.

Common unfairness patterns identified and corrected:

- Requiring a specific HTML element (table vs. list) when the prompt only said 'display a list'
- Marking a visual criterion as failed because sample data was not pre-loaded — when the prompt did not ask for pre-loading
- Stacking two independent behaviours in one criterion, causing a failure cascade when only one broke
- Using forbidden vague words ('correctly', 'properly') that create ambiguous pass/fail conditions

The key discipline: if a rubric item required mental reframing to make the model seem to pass — it needed to be rewritten. The job was to measure the model honestly, not to find ways to give it credit.

◆ WHY THIS MATTERS FOR MODEL QUALITY

The rubrics written in this process are not just evaluation checklists. They become training signals. A model that learns from well-written, atomic, verifiable rubrics learns to produce observable, testable outputs — the foundation of reliable, auditable AI behaviour.

Poorly written rubrics — vague, stacked, or unfair — produce corrupted training signals. The model learns to satisfy the letter of a criterion without achieving its intent. This is how evaluation debt accumulates in AI systems: rubric quality degrades model quality, and the degradation is invisible until the model is deployed.

Good rubric design is, at its core, a form of quality assurance for the AI training pipeline itself.